第二课 密码学基础

信息安全要素

1、机密性

确保信息不被非法获取（保密性）。

常见威胁：窃听、盗取文件、社会工程学（利用人的心理、信任、习惯或疏忽来获取信息或权限）

👉 窃听、偷看、未授权读取、泄露

2、完整性

确保系统或数据被恶意篡改后我们能够发现。

导致完整性破坏的原因：

- 合法用户的失误
- 非法用户的篡改

👉 改写、删改、伪造、插入

3、可用性

系统能以足够的能力、预期的方式、可接受的性能提供服务。

导致可用性破坏的原因：

- 设备故障
- 软件错误
- 环境因素
- 人为攻击

👉 宕机、故障、环境影响、DOS攻击（拒绝服务攻击）

4、非否认性

行为发生后，参与者不能否认自己做过某个操作。

依赖：数字签名、公钥密码体制、证书/PKI、时间戳、审计日志

密码学基本概念

基本术语

- **明文**（plaintext）：需要保护的原始信息
- **密文**（ciphertext）：加密后的结果
- **加密**（encryption）：明文 → 密文
- **解密**（decryption）：密文 → 明文
- **密码体制**：加密算法 + 密钥体系
- **密码编码学**：设计和实现密码
- **密码分析学**：分析和破译密码
- **密码分析**：对密码的破译过程

柯克霍夫斯原则

即便算法公开，只要密钥保密，系统仍应安全

=

密码算法安全性基于密钥，而不是保密相关算法

密码学史上重要事件

1. 1949年，**香农**发表《保密系统的通信原理》 → 密码学理论基础之一
2. 1976年，**Diffie、Hellman** 发表《密码学的新方向》 → **公钥密码学**
3. 1978年，**RSA** 密码体制出现 → **事实上**的标准（非官方但普遍使用）
4. 1978年，**DES** （数据加密标准）

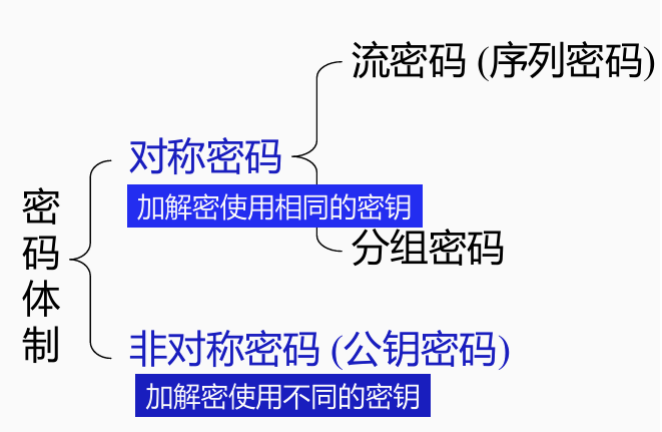
> **Note**
>
> **公钥密码学**
>
> 使用两把不同的密钥进行加密和解密，即**非对称密码学**。
>
> 在1976年以前，几乎所有密码系统都是**对称密码体制**，加密和解密使用同一把密钥。
>
> **RSA & DES**

RSA	DES
公钥密码（非对称密码）	对称密码
1978 RSA算法	1978 DES标准
1992 写入国标 X.509	

密钥体制

五元组（P,C,K,E,D） → 明文空间，密文空间，密钥空间，加密算法，解密算法

加密算法/函数必须**单射**



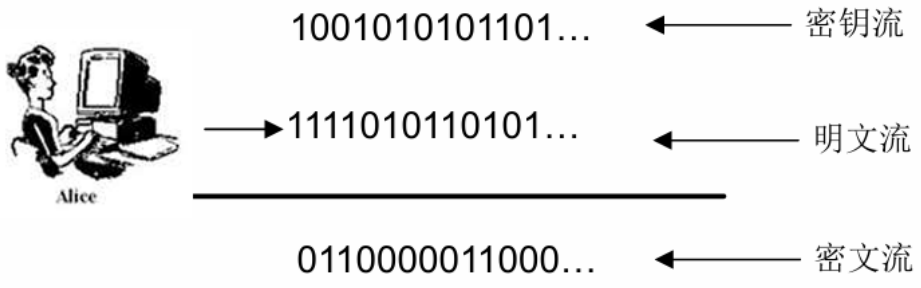
对称密码体制

常见结构

（一）一次一密

理论上最安全，密钥与明文等长，并且每个密钥只能使用一次。

1、加解密简单：对应比特异或



2、特点：

- **真随机序列**，且不重复使用
- 安全性基于密钥流的**随机性**
- 唯一得到证明的**无条件安全**的密码体制

> **Note**
>
> **真随机序列**
>
> - 无规律性（无可预测性） → 无统计特征
> - 均匀分布
> - 不相关性（无记忆性）
>
> **无条件安全**
>
> 完善保密性，不能通过分析密文获得明文的任何信息

3、缺点：

- 实用性不强，无法用于普通互联网通信
- **密钥太长，存储分发困难**
- 密钥流绝不能重复

4、使用场景：

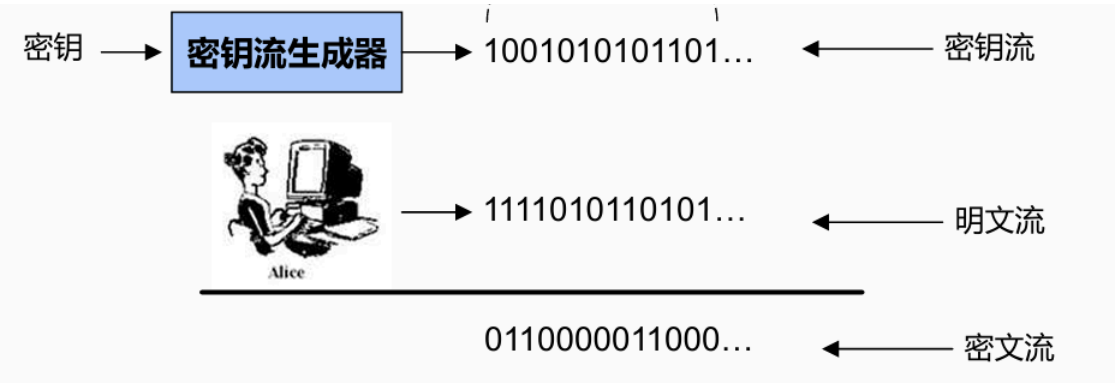
应用于需要无条件安全的**军事或政府领域**

（二）流密码

一次一密 plus版，用一个**较短的密钥（种子seed）**生成密钥流，在攻击者(计算能力是有限的)看来是随机的(**伪随机**的)。

👉 **若为真随机，流密码 = 一次一密**

1、加解密：逐字节异或



2、分类：

- **同步流密码**：密钥流只根据密钥产生，**与明文流无关**
- **自同步流密码**：密钥流不仅与密钥有关，还**与明文流有关**

3、核心：**密钥流生成器**

- **不可预测性**：攻击者不能根据已经看到的密钥流预测后续密钥流，流密码就能在计算意义上保持安全

4、特点：

- 按位加密
- 数据连续输出，**边产生密钥流边加密** → “流”密码
- 速度快，内存占用小，延迟低

5、缺点：

密钥流绝不能重复

6、适用场景

连续数据流场景：网络通信、视频、音频、无线通信

7、常见流密码

- RC4（基本淘汰）
- ChaCha20

（三）分组密码

把明文切成固定长度的分组，然后在密钥控制下，把每个明文分组变成同样长度的密文分组。

🪄 AES 128 bit

1、特点：

- 固定长度，**每块独立/按模式加密**（按块加密）
- 带密钥的复杂置换，常用“混乱”和“扩散”思想
- 最后一个分组**填充**
- 密钥为**较短固定长度密钥**
- 安全性依赖算法设计

2、适用场景

- Hash函数、消息认证码(MAC)的构造模块
- 大量数据：文件、数据库、磁盘、HTTPS

3、常见分组密码

- DES
- AES

> **Important**

一次一密	流密码	分组密码
理论最安全	短密钥生成密钥流	固定长度分组
密钥 = 明文长度	连续逐位加密	每块独立加密
几乎不用	适合实时通信	现代应用最广

流密码 & 分组密码

流密码：速度快、代码量少、适合连续数据流

分组密码：密钥可以重复使用、有标准算法（DES、AES）、适合大量数据、互用性更好

应用	所用密码	评价
数据包	AES	与其他软件的互用性不是问题
E-mail	AES	也可用流密码，但考虑到AES更安全更通用。使用AES可以使你与E-mail服务器保持互用性。
SSL	RC4(流密码)	速度是非常重要的，每次通信需要使用不同的密钥，而且事实上，所有浏览器和服务器都有RC4
文件加密	分组密码	可以用相同的密钥保护不同的文件，密钥管理很方便

DES

Data Encryption Standard 数据加密标准，一种对称密钥 分组密码算法

👉 对称密码 + 分组密码 + **Feistel 结构**（加密和解密过程几乎相同，只需把子密钥顺序反过来即可）

1、特点

- **明文分组 = 密文分组 = 密钥长度 = 64 bit = 有效密钥长度 56 bit + 奇偶校验位 8 bit**
- 安全性：依赖于8个非线性代换S盒
- 雪崩效应（明文或密钥的一点小的变动使密文发生一个 大的变化）

2、优点

- 运算速度快：异或、查表、位运算
- 算法成熟
- 奠定现代分组密码基础

3、缺点

- 密钥太短，随着计算机技术的发展，不足以抵御穷举攻击
- 分组只有64 bit，较小的数据块在处理海量数据时容易出现重复，安全性相对较弱。

AES

Advanced Encryption Standard 高级加密标准，对称密钥 分组密码算法

👉 对称密码 + 分组密码 + **代换-置换网络**（SPN）

1、特点

- 分组长度固定 128 bit
- 密钥长度 = 128 / 192 / 256 bit
- 雪崩效应（明文或密钥的一点小的变动使密文发生一个 大的变化）

2、优点

- 密钥足够长 128bit → 2^{128}次尝试，几乎不可暴力破解
- 现代 CPU 有专门硬件 AES-NI ，运算速度快
- 标准统一，应用广泛

核心特点

1、特点

- 加密、解密使用**相同**的密钥，收发双方必须**事先**共享密钥
- 算法公开，密钥保密
- 运算速度快，适合加密大量数据（大文件、视频、数据库等）
- 适用于**点对点通信**，双方必须提前建立安全的密钥共享机制

2、优点

- 加密速度快 — 运算量小，效率高
- 算法成熟
- 资源消耗少 — 相比公钥密码，占用 CPU 和内存较少，因此广泛应用于实际数据加密
- 安全性高

2、缺点

- 密钥管理困难（密钥爆炸问题）
- 密钥分发困难（安全信道 & 密钥 之 鸡生蛋还是蛋生鸡）
- 无法实现不可否认（双方都使用同一个密钥，无法证明发送身份）

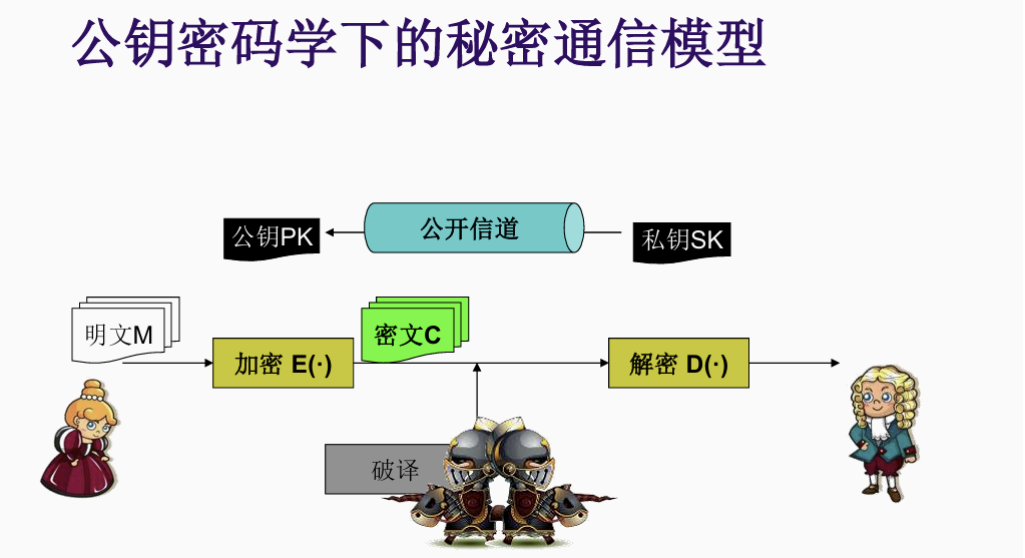
公钥密码体制

👉 加解密使用不同密钥：

- 加密 — 公钥，任何人都可以知道
- 解密 — 私钥，必须保密

1、通信模型

Bob 生成 PK、SK，通过互联网发布公钥 PK（公开信道），Alice获取 PK 后，使用 PK 加密明文发送给 Bob。Bob收到后通过私钥 SK 解密，获取明文。



核心特点

1、特点

- 使用一对密钥，公钥公开，私钥保密
- 算法公开，安全依赖私钥
- 算法需要**陷门单向函数** → 计算容易，求逆困难，知道陷门信息（私钥）求逆容易
- 基于数论**困难假设**：离散对数问题、大整数分解问题、Diffie-Hellman问题

2、优点

- 密钥分发更容易
- 支持**非否认机制**
- 适合：密钥交换、身份认证、数字信封、数字签名

3、缺点

- 计算速度慢，不适合直接加密大量数据
  - RSA 最快情况，也比 DES 慢上100倍
- **密钥长度长** — 为了达到相同安全等级，公钥密码需要更长的密钥

常见算法

（一）Diffie-Hellman 密钥交换协议

两个从未见过面的人，如何实现秘密通信 → 如何在**公开信道**上，实现对称密钥的**秘密传递**。

1、密钥**交换**过程

- 公开两个参数： 大素数 p、生成元 g
- 双方选择私钥： a、b
- 分别计算各自公钥并交换： $A = g^a \mod p$、$B = g^b \mod p$
- 双方计算共享密钥： $Ka = B^a \mod p = g^{ab} \mod p$

  $Kb = A^b \mod p = g^{ab} \mod p$

  $K = Ka = Kb$ = 共享密钥

🥷 攻击者知道 p、g、A、B，但计算a、b是离散对数问题

2、特点

- 安全性基于**Diffie-Hellman问题**：已知 g、g^a、g^b，求 g^{ab}
- 解决密钥分发问题，无需提前共享，可通过公开信道协商出共享密钥
- 只生成密钥，不用于加密数据

3、优点

- 解决 AES 的密钥分发问题
- （相对）安全性高，建立在DH问题、离散对数问题之上
- 广泛应用

4、缺点

不能验证身份，无法确认通信对方的真实身份，容易受到**中间人攻击**

👉 实际应用中，通常结合**数字证书/数字签名**来验证身份

（二）RSA

Rivest-Shamir-Adleman 公钥密码算法，同样可解决密钥分发问题

同等强度下的密钥长度比较：

DES密钥长度(比特)	RSA密钥长度(比特)
56	384
64	512
112	1792
128	2304

1、密钥生成过程

- 选择两个大素数 p、g
- 计算模数 n = pg
- 计算欧拉函数 φ(n) = (p - 1)(q - 1)
- 选择公钥指数 e： e 与 φ(n) 互质，通常 e = 65537
- 计算私钥：d 是 e 关于 φ(n) 的模逆元，$ed \equiv 1 \pmod{\varphi(n)}$

得到公钥 (e,n)，私钥 (d,n)

- 加密：$c = m^e \bmod n$
- 解密：$m = c^d \bmod n$

> **Caution**
>
> **DH & RSA**
>
> DH：共享密钥是双方共同计算得到的，从未在网络上进行传输。
>
> RSA：一方生成共享密钥，通过RSA加密，将密钥传输给另一方。
>
> 👉 协商 vs 分发

2、特点

- 安全性基于**大整数分解困难问题**
- 非对称密钥（公钥+私钥）

3、优点

- 解决密钥分发问题，无需提前共享秘密密钥。
- 支持数字签名，能够实现身份认证、完整性校验、不可否认性

4、缺点

- **运算速度慢**，大整数模幂运算计算量大

软件实现 = DES 1/100

硬件实现 = DES 1/1000

- 密钥较长，通常需要2048位甚至更长
- 不适合直接加密大量数据，一般**只加密对称密钥**

> ⚠ **Warning**
>
> 目前仍无法证明对 RSA 的破译是否等同于大整数分解

（三）ElGamal加密方案

在DH密钥交换上发展而来，

1、特点

- 安全性基于有限域乘法群上的离散对数困难假设
- 随机化加密：密文由明文和随机数r确定，每次加密都重新生成随机数，即**同样明文使用不同随机数，密文也不同**，减少同样明文总是产生同样密文带来的信息泄露

2、优点

- 可扩展为数字签名算法（ElGamal签名算法）
- 安全性较高，基于DH问题和离散对数问题

3、缺点

- 密文长，为明文两倍
- 运算速度慢
- 随机数要求严格，必须真正随机且不可重复

（四）ECC加密方案

Elliptic Curve Cryptography 椭圆曲线密码，一种基于**椭圆曲线离散对数问题（ECDLP）**的公钥密码体制

1、特点

- **安全性高**，基于椭圆曲线离散对数问题的困难性之上。
- 密钥短 → 计算效率高、内存占用少、网络传输数据更少
- 适合资源受限设备：手机、智能手表、物联网设备

2、与 RSA 性能对比

相同安全等级下：

安全等级	ECC 密钥	RSA 密钥
约 128 位安全性	256 bit	3072 bit
约 192 位安全性	384 bit	7680 bit
约 256 位安全性	512 bit	15360 bit

👉 ECC 用几百位密钥，就能达到 RSA 几千位密钥的安全效果。

原因：ECC 所依赖的数学难题（ECDLP）比 RSA 所依赖的大整数分解问题更难。**在目前已知的最佳攻击算法下，要达到相同安全等级，破解 ECDLP 所需的计算量更大。**因此，RSA需要用更长的密钥来保证安全性。

> **Important**
>
> 约128 bit位**安全性**
>
> 非密钥长度，而是破解该算法所需的工作量相当于穷举一个 128 位对称密钥。

3、与RSA安全性对比

- **RSA** 的安全性依赖于**大整数分解问题**。
- **ECC** 的安全性依赖于**椭圆曲线离散对数问题**。

在相同安全等级下，ECC 所需密钥长度远小于 RSA，因此**安全效率更高**。

哈希函数

散列函数、杂凑函数、单向函数，将任意长度的**消息**映射成固定长度**散列值**的函数，**压缩**功能

- 输入：消息
- 输出：散列值 / Hash 值 / 消息摘要

🪄 散列值长度不受输入影响

```
 hello
    ↓
 SHA-256
    ↓
185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

输出固定为256 bit（64个十六进制字符）
```

1、特点

- 无论输入多长，输出长度固定
- 雪崩效应：改变消息的一个/几个 bit 都会使散列值发生改变
- 单向性：无法反推
- **碰撞**：不同的消息得到了相同的散列值，优秀的Hash 应该极难发生碰撞

2、分类

- 单向 Hash 函数 ： 原像稳固（给定散列值，很难倒推出原文）

  👉 **反推困难**：用户密码存储、消息摘要
- 弱抗碰撞 Hash 函数：第二原像稳固（已知一个原文，很难找到另一个同摘要的消息）

  👉 **内容不容易被替换**：文件完整性校验、数字签名
- 强抗碰撞 Hash 函数：碰撞稳固（很难找到两条散列值一样的消息）

  👉 **防止制造碰撞文件**

3、性质

密码学上安全的：

- 有效性：容易计算
- 确定性：对于相同消息，总得到相同的散列值
- 单向性
- 抗碰撞性
- 随机性：输出是随机的（理想）

> **Note**
>
> **哈希函数不能既是确定性的，又是真正随机的**
>
> 随机性只是理想情况，Hash 的输出应该"看起来像随机数"，但实际上仍然是确定计算出来的。 → 伪随机 → 不可区分性（ 不存在任何有效算法能将它的输出与随机输出相区分）
>
> 理想的Hash函数不存在 → 随机谕言器

迭代Hash函数

迭代：重复应用压缩函数，进行多次压缩，最后得到散列值

相关算法

1、MD5

1992，Rivest

输入：消息分为 512 bit 一组处理

散列值长度：128 bit

特点：

- 直接构造法：不依赖任何密码模块和假设条件
- 算法简洁，计算速度快
- 早期用于商业文件校验、密码存储

缺点：**已存在碰撞攻击**，不再适用于数字签名、数字证书等安全场景

使用场景：非安全用途文件校验、数据去重等

> **Note**
>
> 2004年，在CRYPTO'04学术会议上，王小云首次公开提出了 **MD5 的碰撞攻击方法**，证明可以在可行时间内构造出两个不同的消息，使它们具有相同的 MD5 哈希值。

2、SHA

SHA-1

输入：消息分为 512 bit 一组处理（与MD5相同）

散列值长度：160 bit

特点：

- 运行速度比 MD5 慢
- 安全性比 MD5 高

缺点：**已存在碰撞攻击**

> **Note**

> 2005年，王小云团队公布了 **SHA-1 的理论碰撞攻击方法**，大大降低了寻找 SHA-1 碰撞的理论复杂度。
>
> 2017年， **SHAttered** 项目实现了**世界上首次公开的 SHA-1 实际碰撞**，进一步证明 SHA-1 已不再安全。

SHA-2（目前主流）

目前应用最广泛

系列：

- SHA-224：散列值 224 bit
- SHA-256：散列值 256 bit（64个 十六进制字符），最常见
- SHA-384
- SHA-512

完整性校验

消息认证码 MAC

基于对称密钥生成的一段固定长度的认证信息，用于验证消息在传输过程中**是否被篡改**，以及**消息是否来自拥有共享密钥的一方**

$$MAC = F(K, M)$$

K：共享密钥　　M：消息

发送方把 **消息 + MAC** 一起发送给接收方，接收方利用**相同的密钥重新计算 MAC**，若结果一致，则说明消息未被篡改且来自知道该密钥的发送方。

1、特点

- 采用对称密钥体制，发送方和接受方共享同一把密钥
- 验证消息完整性
- 验证消息来源
- **不能提供不可否认性**

  👉 双方都知道相同密钥，接受方可伪造消息来自发送方，发送方可以否认曾发送过消息

2、常见算法

- HMAC （Hash-based MAC）：最常见，基于哈希函数

- CMAC：基于AES的消息验证码

数字签名

基于公钥密码体制的一种认证技术

发送方利用自己的**私钥**对消息（通常是消息摘要）进行签名，接收方利用**发送方的公钥**验证签名是否正确。

流程如下：

```
发送方：                          接收方：
消息                              消息          数字签名
  ↓                                 ↓             ↓
Hash                              Hash          发送方公钥验证
  ↓                                 ↓             ↓
消息摘要                          得到摘要①      得到摘要②
  ↓
发送方私钥签名                    比较①和② ，相同即验证成功
  ↓
数字签名
```

1、特点

- 采用公钥密码体制
- 速度相比 MAC 较慢
- **抗伪造性**：任何人都不能伪造签名者的合法签名
- 不可重用性：一个消息的数字签名不能用于其他消息
- 保证消息完整性
- 验证发送者身份：公钥是公开的，任何人都可以验证签名的合法性
- **具有不可否认性**

  👉 只有发送者拥有自己的私钥，不能否认自己签过名。

2、常见算法

RSA 签名

私钥签名 + 公钥验证，**基于大整数分解困难问题**

❗ 但纯 RSA 签名方案存在漏洞

👉 利用 Hash 函数改进 RSA 方案

对消息M签名 → 对 H（M）签名

作用：抵抗存在性伪造（hash的单向性和抗碰撞）

加快计算速度（消息很长时，先用Hash压缩）

抵抗利用两个消息的签名产生新消息的签名$(H(m1)H(m2))^d \neq H(m1m2)^d \pmod n$

应用：证书、TLS、文档签名

ElGamal签名

基于 **离散对数问题**（DLP） 的困难性，是后续 DSA 的重要理论基础。

私钥签名 + 公钥验证

❗同一私钥下，随机数 k 绝不能重复使用。

如果两次签名使用了同一个 k，击者可以计算出私钥 x。

应用：DSA、理论研究

补充内容

数字信封

取长补短，利用公钥密码安全地传输对称密钥，再利用对称密码加密真正的数据

1、核心思想

- 随即生成一个对称会话密钥
- 用对称密钥加密明文数据
- 用接收者公钥保护该对称会话密钥
- 接收者先用私钥恢复会话密钥，再用会话密钥解密正文

👉 对称密码快，适合加密大量数据

非对称密码速度慢，但可以解决密钥分发问题

2、特点

- 利用各自优点，混合加密
- 既保证安全性，有保证高效性

Lamport（S/Key）—— 一次一密

一种经典的**一次性口令**（One-Time Password，OTP）认证方案，利用**哈希链**实现身份认证，不需要在网络上传输真正的密码，因此能够有效抵抗密码窃听和重放攻击。

1、核心思想

一次性口令：一次登录使用一个密码，使用过后立即失效，下次登录使用新的密码。

2、基本原理

- 对初始密码 P 进行连续 hash，生成Hash链
- 初始：服务器只保存 $H^n(P)$
- 第一次登陆，用户发送 $H^{n-1}(P)$，服务器验证 $H(H^{n-1}(P)) ?= H^n(P)$
- 认证成功，服务器将保存值更新为 $H^{n-1}(P)$

3、特点

- 每个口令只能使用一次，认证成功后立即失效。
- 安全性依赖于 Hash 函数的**单向性**，攻击者无法由当前口令推算出下一次要使用的口令。

4、缺点

必须保持**同步**，可能因为不可靠信道/死机 出现丢失同步的情况

门限秘密共享

一种将**一个秘密分成多个份额**（Share）进行保存的技术，只有达到规定数量的参与者合作，才能恢复秘密；少于这个数量，则无法获得秘密。

👉 Shamir 门限秘密共享方案

1、核心思想

(k, n) 门限秘密共享方案：

- 将一个秘密 S，将它分成 n 份，分别交给 n 个人保管。
- **至少 k 个人合作**，才能恢复秘密。
- **少于 k 个人**，无法恢复秘密。

2、特点

- 提高安全性，秘密不会集中存放
- 容错性强，部分份额丢失仍可恢复秘密

- 保证保密性，少于门限数量无法恢复秘

3、应用

- 密钥备份与应用
- 数字钱包（加密货币私钥管理）
- 企业重要数据管理
- 多人共同授权系统